

地址翻译与虚拟存储硬件

虚拟地址怎样成为带权限、可供物理存储层次消费的访问身份

408 · 计算机组成原理 Topic CO-07

母问题：程序地址怎样经过映射、驻留/有效性与权限检查，形成合法物理访问？

一次页式地址访问可压成

$\text{Request}(VA, type, privilege) \rightarrow \text{VPN} / \text{Offset} \rightarrow \text{TLB or Page Walk}$
 $\rightarrow \text{Mapping} / \text{Permission Check} \rightarrow \text{PA or Fault}$

箭头表示**翻译依赖与合法性检查**，不是固定时钟流水。实现可以并行查 TLB/Cache，但最终访问必须有合法映射与权限依据。

本册负责什么

- **Own:** VA/PA 拆分、VPN/PFN、页内偏移不变量、PTE 硬件可见字段、TLB、page walk、多级页表、地址空间标签/失效、PIPT/VIPT/VIVT 的翻译接口，以及硬件可观察 fault 边界。
- **Use:** CO-06 提供 Cache 组织，OS-04 提供页框分配、驻留、置换、COW 和 fault repair；X-B02 负责软硬件交接。
- **Stop Boundary:** 得到合法 PA 后若开始模拟 Cache 状态，转 CO-B02/CO-06；fault 后若开始分配 frame、I/O、阻塞或置换，转 X-B02/OS-04。

目录

1 术语先行：每个对象只回答一个问题	3
2 分页的核心不变量：只替换页身份，不改变页内位置	3
3 PTE 与页表：映射不是单独一个 PFN	3
3.1 线性页表与多级页表	4
4 Page Walk：地址依赖链，不是固定“几次 DRAM”	4
5 TLB：把近期翻译结果缓存起来	4
5.1 全相联与组相联 TLB	4
6 一次翻译的状态轨迹：Hit、Miss、Fault 分层	5
7 翻译副本必须与页表当前承诺一致	5

8 TLB 与 Cache 的接口：哪些位可以并行使用	6
8.1 PIPT	6
8.2 VIPT	6
8.3 VIVT	6
9 分段与段页式：不是分页字段的另一个名字	6
10 翻译成本：EAT 是路径期望，不在本册复制 AMAT 公式	6
11 贯穿母例：一次 Load 的三类慢路径	7
12 概念边界	7
13 做题控制协议	7
14 本册与其他 Owner 的接口	8

1 术语先行：每个对象只回答一个问题

核心术语

- **VA (Virtual Address)**: 程序/指令发出的地址，属于当前地址空间。
- **PA (Physical Address)**: 翻译成功后交给物理存储层次/设备接口的地址身份。
- **VPN / PFN**: 虚拟页号与物理页框号；分页翻译主要替换页身份。
- **Page Offset**: 页内位置；合法固定页大小翻译中保持不变。
- **PTE**: 描述页映射、present/valid、权限以及架构规定状态的页表项。
- **TLB**: 缓存近期 VPN→PFN 与权限等翻译结果的 translation cache。
- **Page Walk**: TLB miss 后按页表层级读取 PTE、逐步得到叶项的过程。
- **Page Fault**: 页式访问无法沿普通快路径完成时产生的同步异常入口；原因可进一步区分 non-present、permission/protection 等。408 经典“缺页异常”通常特指页面当前不驻留的分支。
- **ASID / PCID**: 地址空间标签，用于区分不同地址空间中相同 VPN 的翻译副本；具体名称和语义服从 ISA。

2 分页的核心不变量：只替换页身份，不改变页内位置

若 page size 为 2^p B、按字节编址，则

$$VA = [VPN \mid Offset_p], \quad PA = [PFN \mid Offset_p].$$

合法映射下

$$VA[p-1:0] = PA[p-1:0].$$

因此页内偏移是分页翻译最重要的稳定量：页表只需要为 VPN 提供新的 PFN/权限，块内低位、某些 Cache index 位等若完全落在 page offset 中，就不需要等待 PFN 才能确定。

3 PTE 与页表：映射不是单独一个 PFN

PTE 通常至少要表达“当前地址是否能继续翻译、映射到哪里、允许怎样访问”。常见硬件可见信息包括：

字段类别	作用	做题后果
present / valid	当前是否存在硬件可用映射/驻留页框	为 0 时进入 non-present / 缺页类 fault
PFN	当前物理页框身份	与 page offset 拼接形成 PA
R/W/X、 privilege	当前访问类型是否允许	失败进入 protection/permission fault
accessed / reference	记录近期访问状态（若架构提供）	可成为 OS replacement 的硬件输入
dirty / modified	记录页面是否被写过（若架构提供）	可影响 OS 换出写回
架构扩展位	cacheability、global、large page 等	只按题设/ISA 使用，不跨架构硬套

若 PTE 不 present，其中某些位可能由 OS 解释为后备信息或软件状态；不能无条件把“PFN 字段”当成仍然有效的物理页框号。

3.1 线性页表与多级页表

若 VA 有 v 位、page size 为 2^p B、PTE 大小为 e B，线性页表理论大小为

$$2^{v-p} \times e.$$

多级页表的主要动机是让稀疏地址空间只为实际需要的页表子树付空间成本。若一张页表页能容纳

$$N = \frac{PageSize}{PTESize}$$

个表项，则完整索引层可提供 $\log_2 N$ 位索引；顶层不一定恰好占满。

4 Page Walk：地址依赖链，不是固定“几次 DRAM”

典型多级 walk 的依赖是

$$root + index_1 \rightarrow PTE_1 \rightarrow next\ table + index_2 \rightarrow \cdots \rightarrow leaf\ PTE \rightarrow PFN.$$

后一级页表地址依赖前一级 PTE 返回的下一层基址，因此逻辑依赖必须串起来；但 PTE 本身可能命中 CPU Cache，硬件也可能并行/预取部分信息，所以“ L 级页表”不能无条件改写成“固定 $L+1$ 次 DRAM 访问”。时间题应把 page walk 的每个访问对象交给 CO-06 的存储层次成本模型。

5 TLB：把近期翻译结果缓存起来

TLB 保存 translation entry，而不是程序数据。TLB hit 表示当前请求所需 VPN 的翻译副本和必要权限状态可用；TLB miss 只表示翻译缓存缺项，page walk 仍可能成功，因此 TLB miss 不等于缺页，也不等于 Page Fault。

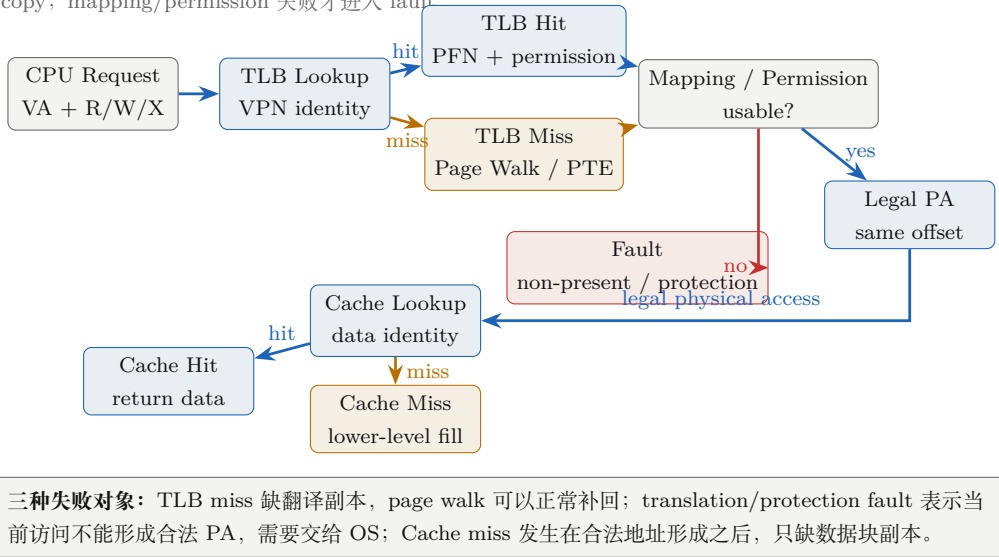
5.1 全相联与组相联 TLB

若 VPN 有 n 位，TLB 有 $S = 2^s$ 组，则经典组相联组织可把 VPN 低 s 位作为 set index，其余 $n-s$ 位作为 tag；全相联相当于只有一个 set、没有 set index。题面另给 ASID/PCID 时，地址空间标签也参与翻译身份判定。

若 page size 不变、TLB 组数不变，而 VA 宽度增加 k 位，则 VPN/tag 相应增加 k 位；PFN 位数仍由 PA 宽度和 page size 决定，不能因 VA 变宽自动增长。

6 一次翻译的状态轨迹：Hit、Miss、Fault 分层

一次 Load 的三条轨道：翻译副本、权威页表、数据副本必须分流
分页只替换页身份： $VA = [VPN \mid offset]$ ，成功时 $PA = [PFN \mid offset]$ 。TLB miss 只缺 translation copy；mapping/permission 失败才进入 fault



实现可把 VIPT Cache index 与 TLB lookup 并行；本图箭头表示语义依赖，不表示必须逐框串行等待。

图中的主线表示语义依赖而不是固定串行时序：TLB hit 可跳过 page walk；只有形成合法 PA 后才进入普通 Cache hit/miss；VIPT 等实现还可能让部分 Cache 定位与 TLB lookup 并行。关键是区分当前失败对象：

事件	当前缺失/失败对象	下一动作	是否必进 OS
TLB miss	翻译缓存副本	page walk / refill	不一定
缺页（non-present/residency fault）	当前可用驻留页框/映射	fault handoff, OS 修复或拒绝	是
Permission / protection fault	本次 R/W/X 或 privilege 不允许	fault handoff	是
Cache miss	合法地址对应的数据块副本	下一级取块/refill	通常否

真实 ISA 可能把 non-present 与 protection 编码为同一个 page-fault exception 家族；408 经典题若只讨论“缺页”，优先按“页面当前不在主存/不可直接使用”理解。

7 翻译副本必须与页表当前承诺一致

相同 VPN 在不同地址空间可以映射不同 PFN；如果 TLB 只看 VPN 而不区分地址空间，就可能产生 homonym。ASID/PCID、切换时 flush/invalidate 等机制用于维护地址空间隔离。

OS 修改 PTE 后, 旧 TLB entry 不会凭空自动消失。系统必须按 ISA 的失效和排序规则, 保证后续翻译不会继续消费已经失效的旧映射。408 简化模型中常表现为: 页被换出、PTE present 清零后, 对应旧 TLB entry 不能继续保持可命中的有效状态。

8 TLB 与 Cache 的接口: 哪些位可以并行使用

8.1 PIPT

PIPT 的概念路径是

$$VA \rightarrow TLB/PageWalk \rightarrow PA \rightarrow Cache.$$

身份最直接, 但 Cache lookup 往往依赖翻译结果。

8.2 VIPT

VIPT 利用 page offset 在 VA/PA 中不变: 可先用页内低位索引 Cache, 同时用 VPN 查 TLB, 得到 PFN 后再用物理 tag 完成身份确认。若 page offset 为 p 位、block offset 为 b 位, 则经典安全 index 位数至多为 $p - b$, 即

$$sets \times block\ size \leq PageSize.$$

等价地, 对 E 路 Cache, 每一路数据容量不超过 page size。这个约束来自位预算, 不是固定容量口诀。

8.3 VIVT

VIVT 直接按 VA 做 Cache 身份, 会引入不同地址空间相同 VA (homonym) 和多个 VA 指向同一物理数据 (synonym) 等问题; 具体处理依实现, 408 若题面未要求, 不自行补机制。

9 分段与段页式: 不是分页字段的另一个名字

分段把地址表示为 $(segment, offset)$, 段表项提供 base、limit 和保护属性。硬件先完成段存在/界限/权限检查, 再计算

$$PA = base + offset.$$

段长可变, 因此不能像分页那样把物理地址写成“替换高位、保留固定低位”的简单拼接。

段页式可写成 $(segment, page, offset)$: 先由段表得到该段页表基址与界限, 再由 page index 查页表得到 PFN, 最后拼接页内 offset。没有翻译缓存、且表项访问都真正落到主存时, 经典逻辑访问数才可按“段表 + 页表 + 数据”计; 若表项可被缓存, 实际等待时间仍回到存储层次路径计算。

10 翻译成本: EAT 是路径期望, 不在本册复制 AMAT 公式

地址翻译性能取决于 TLB hit/miss、page-walk 层数、PTE 命中在哪一级 Cache、TLB/Cache 是否并行, 以及 fault 后是否进入 OS 慢路径。本册只提供这些路径事件; 平均时间统一调用 CO-06 《存储层次与 AMAT》的条件期望模型。

因此:

- TLB hit rate 高通常能减少 page-walk 路径，但不等于数据 Cache hit；
- 多级页表主要解决页表空间，不天然降低每次翻译延迟；
- Page Fault 的 OS/I-O 修复成本不能静默塞进普通 Cache miss penalty；
- AMAT/EAT 题先写路径，再决定哪些访问串行、并行或已被缓存。

11 贯穿母例：一次 Load 的三类慢路径

对一次 'load [VA]':

1. 先由 page size 拆 VPN/offset；
2. 若 TLB hit，检查翻译与权限并形成 PA；
3. 若 TLB miss，page walk 读取 PTE；walk 成功则 refill TLB 并形成 PA；
4. 若页面不驻留，产生经典缺页 fault，交给 OS；若权限不允许，产生 protection/permission fault；
5. 只有形成合法 PA 后，数据访问才交给 Cache；Cache 是否 hit 是另一层判断；
6. fault 若可修复，OS 更新必要状态并按架构规则 retry 原访问；若不可修复则拒绝/终止；
7. 最终 load 只有在无异常完成路径上才提交目标数据。

12 概念边界

边界	为什么容易混淆	真正判据 / 题面信号	混淆后果
VPN $\neq$ PFN	都是页号高位	虚拟页身份 vs 物理页框身份	错拼 PA
PTE $\neq$ TLB entry	都含 PFN/权限	权威映射结构 vs 翻译缓存副本	忽略失效/同步
TLB miss $\neq$ 缺页	都让翻译快路径失败	缺 translation copy vs 页面不驻留	无条件进 OS/磁盘
缺页 $\neq$ Permission fault	都可能触发同步异常	residency/present vs access permission	错写 fault 原因
Page Fault $\neq$ Cache miss	都可能很慢	翻译/驻留/权限异常 vs 数据副本缺失	把硬件 miss 写成 OS 调页
Page resident $\neq$ TLB hit	常一起出现	页在主存不保证翻译副本在 TLB	把 page walk 当缺页
VA $\neq$ PA	都是地址位串	程序视图身份 vs 物理存储身份	未翻译就按物理 Cache 定位

13 做题控制协议

1. **Request:** 写 VA、R/W/X、privilege、访问大小；
2. **Geometry:** 由 page size 锁定 page offset，得到 VPN/PFN 位数；

- 3. **TLB**: 确定全相联/组相联、set/tag、当前 entry 状态;
- 4. **Walk**: TLB miss 后按页表层级追踪 PTE, 不直接写 Page Fault;
- 5. **Check**: 区分 present/residency、permission/protection;
- 6. **Output**: 成功时只替换页身份并保留 offset, 形成合法 PA;
- 7. **Handoff**: PA 交给 CO-B02/CO-06; fault 交给 X-B02/OS-04;
- 8. **Cost**: 若问平均时间, 回《存储层次与 AMAT》按真实路径计算。

第一动作

看到虚拟地址题先问: “当前缺的是 translation copy、页面驻留/映射, 还是访问权限? Page Offset 是否保持不变?” 不先回答这两个问题, 就不要把 TLB miss、缺页和 Cache miss 混成一次 “没找到”。

14 本册与其他 Owner 的接口

相邻 Owner	CO-07 输出/使用什么	转交边界
CO-06 Cache	使用 Cache 组织; 输出合法 PA/权限与 page-offset 不变量	开始模拟 data block hit/miss
CO-B02 Translation × Cache	输出 VA/PA 位与翻译状态	开始讨论串并行、VIPT handoff
OS-04 / X-B02	输出 fault 原因与可重试现场	开始分配 frame、COW、page-in、置换与阻塞
CO-05 主存	page walk/PTE 最终调用存储服务	开始推 DRAM/Bank/总线内部成本

一页压缩

VA Request → VPN / Offset → TLB or Walk → Mapping + Permission → PA or Fault

分页最稳定的不变量是 **Page Offset 保持**; 最重要的边界是 **TLB miss、缺页/保护 fault、Cache miss 分属不同对象和处理者**。